

Módulo “Valoraciones y reseñas de escenarios”

Examen Práctico Integrador — PDF 7: Nuevo módulo / funcionalidad

Proyecto: Lugares Interactivos — Visualización interactiva de escenarios deportivos a nivel nacional

App: Expo / React Native (expo-router, Supabase, react-query, react-hook-form + Zod)

Rama: ExamenFinalNicolasReguerin

1 de septiembre de 2026

1. Link del repositorio

github.com/Joaquinf87/proyecto-movil/tree/ExamenFinalNicolasReguerin

2. Funcionalidad implementada

Los usuarios autenticados pueden calificar cada escenario deportivo con **1 a 5 estrellas** y dejar un comentario opcional. La aplicación permite:

- Ver el **promedio** y la **cantidad** de valoraciones de cada escenario (en el detalle y en las tarjetas del catálogo).
- Ver la **lista de reseñas** con el autor y su comentario.
- Gestionar desde la pestaña *Valoraciones* las **mis valoraciones** (consultar, editar o eliminar).

3. Problema o necesidad que resuelve

La app ofrece información oficial de infraestructura deportiva (capacidad, horario, fotos, eventos), pero el usuario no disponía de información sobre la **experiencia y el estado percibido** de cada escenario. Las valoraciones agregan información comunitaria que ayuda a otros ciudadanos a decidir y proporciona retroalimentación al personal de gestión.

4. Usuarios involucrados

- **Ciudadanía** (rol **asistente** / cualquier usuario autenticado): valora, consulta y gestiona sus propias reseñas.
- **Gestores y administradores**: consultan las valoraciones de los escenarios que administran. El **admin** puede eliminar cualquier valoración (moderación).

5. Objetivo

Permitir que la comunidad califique los escenarios y consultar el promedio de forma centralizada, integrada a la navegación existente, sin duplicar datos ni romper la lógica actual.

6. Flujo de funcionamiento

Usuario

1. Abre el detalle de un escenario (o la pestaña Valoraciones)
2. Toca "Valorar" / "Editar mi valoración"

3. Selecciona estrellas (1-5) y escribe un comentario opcional
4. Se validan los datos (react-hook-form + Zod)
5. Se guarda en Supabase (upsert por usuario + escenario)
6. Se recalculan promedio y lista (react-query)
7. Consulta / edita / elimina desde "Mis Valoraciones" o el detalle

7. Pantallas creadas y modificadas

Pantalla / componente	Tipo	Descripción
src/app/(tabs)/ratings.tsx	Nueva	Mis Valoraciones: lista con crear/editar/eliminar
src/app/(tabs)/_layout.tsx	Modificada	Pestaña Valoraciones (icono star-half)
src/app/scenario/[id].tsx	Modificada	Sección Valoraciones: promedio, botón Valora, reseñas
components/rating/RatingStars.tsx	Nueva	Selector / visualización de estrellas
components/rating/RatingFormModal.tsx	Nueva	Modal crear/editar valoración (RHF + Zod)
components/rating/RatingList.tsx	Nueva	Lista de reseñas con acciones propias
components/common/ScenarioCard.tsx	Modificada	Badge de promedio y cantidad
hooks/useScenarioRatings.ts	Nueva	CRUD + estadísticas + caché offline
utils/ratingDraft.ts	Nueva	Persistencia local (borrador + caché)

8. Base de datos: Supabase / PostgreSQL

8.1. Tabla nueva: public.scenario_ratings

Migración 011_create_scenario_ratings.sql.

Campo	Tipo	Notas
id	UUID PK	gen_random_uuid()
scenario_id	UUID FK → scenarios.id	ON DELETE CASCADE
user_id	UUID FK → profiles.id	ON DELETE CASCADE
rating	SMALLINT	CHECK (rating BETWEEN 1 AND 5)
comment	TEXT	default "", máx. 500 (validación en app)
created_at / updated_at	TIMESTAMPTZ	trigger automático

Restricción única: UNIQUE (user_id, scenario_id) – una valoración por usuario y escenario. **Índices:** por scenario_id y por user_id.

8.2. RLS (Row Level Security)

Operación	Política
SELECT	autenticados y anónimos (el público puede ver reseñas)
INSERT	user_id = auth.uid()
UPDATE	user_id = auth.uid()
DELETE	user_id = auth.uid() o rol admin

8.3. Función: `public.scenario_rating_stats()`

Devuelve (`scenario_id`, `average`, `count`) agrupado, usada para el promedio en las tarjetas del catálogo (`SECURITY DEFINER`, solo expone agregados).

9. Operaciones CRUD

Operación	Dónde	Cómo
Create	detalle / modal	<code>upsert</code> con <code>onConflict: user_id, scenario_id</code>
Read	detalle y pestaña	<code>SELECT</code> con join a <code>profiles</code> y <code>scenarios</code>
Update	modal de edición	<code>upsert</code> (misma clave)
Delete	detalle y pestaña	<code>DELETE</code> por id (propia; admin: cualquiera)
Aggregate	tarjetas del catálogo	<code>rpc('scenario_rating_stats')</code>

La UI refleja los cambios al instante invalidando queries de react-query (`['scenario-ratings', id]`, `['my-ratings', userId]`, `['rating-stats']`).

10. Persistencia utilizada

10.1. Supabase (centralizada)

Promedios, reseñas y quién las emitió deben ser consultables por todos los usuarios desde cualquier dispositivo y moderables por `admin` → se almacenan en PostgreSQL vía Supabase.

10.2. AsyncStorage (local) – `utils/ratingDraft.ts`

- **Borrador de la valoración** (`rating_draft:v1:{userId}`): si el usuario escribe estrellas / comentario y cierra el modal sin guardar, el borrador se restaura la próxima vez.
- **Caché offline de “Mis Valoraciones”** (`my_ratings_cache:v1:{userId}`): la pestaña muestra los datos guardados del dispositivo antes de la respuesta del servidor.

Justificación: información privada por dispositivo y temporales de edición. El proyecto ya tenía `AsyncStorage` instalado pero sin uso; es la primera funcionalidad que lo aprovecha.

11. Validaciones y manejo de errores

- **Validación de campos:** Zod en el formulario – calificación requerida entre 1 y 5; comentario opcional con máximo 500 caracteres.
- **Datos inválidos:** el `CHECK (rating BETWEEN 1 AND 5)` y el `UNIQUE` refuerzan a nivel de base de datos.
- **Información inexistente:** `EmptyState` (“Sin valoraciones aún”) y estado de error con reintento.
- **Error al guardar:** mensaje en el modal sin cerrar el formulario.
- **Error de consulta:** `ErrorState` con botón “Reintentar” en la pestaña.
- **Doble envío:** botón deshabilitado con *spinner* mientras guarda.

12. Pruebas realizadas

#	Prueba	Resultado
1	Acceso al módulo (pestaña Valoraciones + botón en detalle)	✓
2	Ingreso de datos válidos (5 estrellas + comentario)	✓ inserción correcta
3	Ingreso de datos inválidos (<code>rating = 6</code>)	✓ rechazado por CHECK
4	Validación de campos (0 estrellas al publicar)	✓ error en UI
5	Guardado de información (CREATE vía <code>upsert</code>)	✓
6	Consulta (lista con <code>profiles</code> + promedio por RPC)	✓
7	Modificación (UPDATE del propio <code>rating</code> / comentario)	✓
8	Eliminación (DELETE propia)	✓
9	Persistencia (caché AsyncStorage + borrador)	✓ implementada
10	Manejo de errores (RLS ajeno bloqueado: INSERT/UPDATE/DELETE)	✓
11	RLS: <code>admin</code> puede eliminar valoración ajena	✓
12	Estadísticas agregadas (<code>scenario_rating_stats</code>)	✓

Las pruebas 1–12 se ejecutaron contra **Supabase local** (script de *smoke test* con `@supabase/supabase-js`): autenticación con usuarios *seed*, CRUD completo, *constraints* y control de RLS.

13. Desafío adicional

- Ordenamiento de reseñas por más recientes (`created_at desc`).
- Edición *inline* vía el mismo modal reutilizado (crear/editar/eliminar desde detalle y desde la pestaña).
- Promedio agregado en tarjetas del catálogo sin duplicar consultas.
- Caché offline de “Mis Valoraciones” y borrador en AsyncStorage.

14. Dificultades encontradas

1. **Privilegios de tabla en Supabase local:** las tablas nuevas no recibían los grants automáticos (`authenticated` sin `INSERT/SELECT`); se agregaron `GRANT` explícitos en la migración.
2. **Caché de esquema de PostgREST:** tras aplicar la migración la API devolvía 404 hasta recargar el esquema (resuelto con `db reset` y reinicio del contenedor `rest`).
3. **Ambigüedad en el listado de migraciones local** que sugería que 011 estaba aplicada cuando no lo estaba; verificado directamente con `psql`.
4. **Tipos de expo-router desactualizados** (`.expo/types/router.d.ts`): errores de TS preexistentes en `manage.tsx` que se regeneran con `expo start`.
5. **Lint preexistente roto** en el repo (ESLint v10 requiere *flat config* y el repo usa `.eslintrc.js`); se aplicó Prettier y `tsc` validado sobre los archivos del módulo.

15. Cómo compilar el APK para Android

La aplicación se ejecuta desde el proyecto nativo `android/` de Expo (SDK 57). Para correrla en un **celular conectado por USB** a la PC:

1. Conectar el celular por USB y habilitar la *depuración por USB*.
2. Verificar la IP de la base: si se usa **Supabase local**, apuntar en `.env` a la IP de la máquina en la LAN (p. ej. `http://192.168.x.x:54321`) y la *anon key* local. En emulador Android usar `http://10.0.2.2:54321`.
3. Compilar e instalar directamente en el dispositivo conectado:

```
pnpm android
```

Este comando (`expo run:android`) compila, instala y lanza la app en el celular.

4. Para generar solo el APK firmado (instalable manualmente), compilar en modo **release** (firma con la *debug key*):

```
cd android
./gradlew assembleRelease
```

El APK generado queda en `android/app/build/outputs/apk/release/app-release.apk` y se instala con:

```
adb install app-release.apk
```

Al acceder por primera vez iniciar sesión con el usuario *seed*: `asistente@test.com / password123`.

16. Evidencias (capturas)

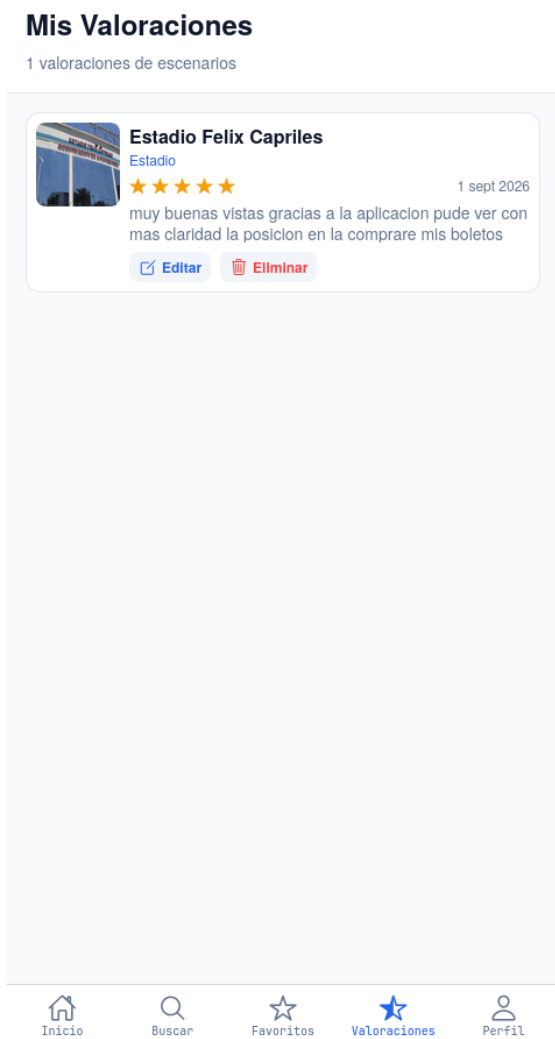


Figura 1: Pestaña *Valoraciones* (pantalla principal del módulo).



Figura 2: Modal *Valorar escenario*: estrellas 1-5 + comentario.



Figura 3: Sección Valoraciones en el detalle: promedio y reseñas con autor.

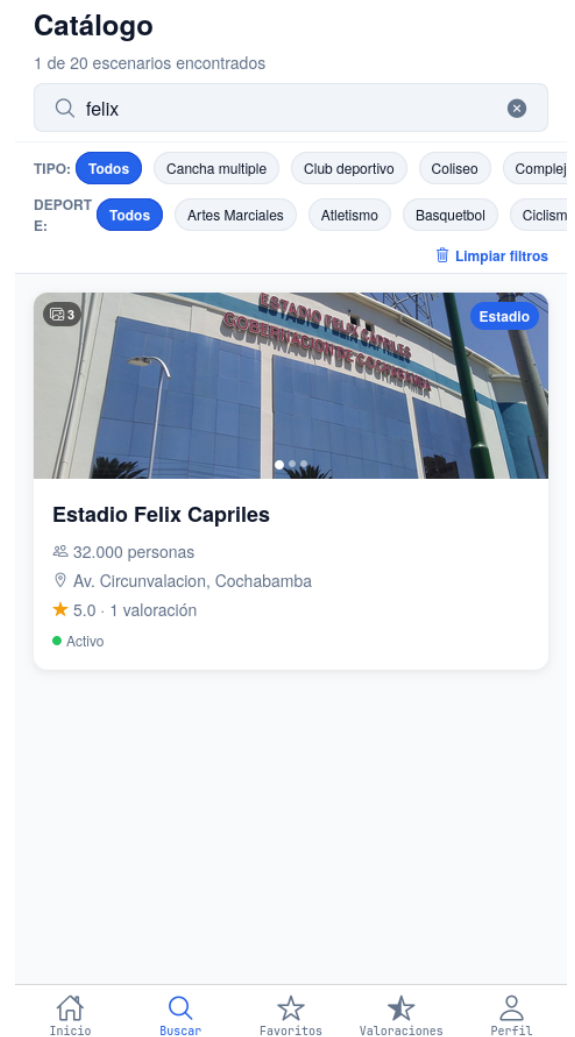


Figura 4: Tarjeta del catálogo con badge de promedio y cantidad.



Figura 5: Validación de campos (error al publicar sin estrellas).



Figura 6: Confirmación de eliminación de una valoración.



Arena Magma Fest

Coliseo

Descripción

Espacio multiuso para eventos deportivos, musicales y de entretenimiento en la zona este de la ciudad.

Información

 CAPACIDAD

6.000 personas

 DIRECCIÓN

Av. Principal, Equipetrol, Santa Cruz de la Sierra

 HORARIO

Lun-Dom 10:00-23:00

 Basquetbol

 Voleibol

 Artes Marciales

★ Valorar

★

 Guardado

Figura 7: Acceso al módulo desde el detalle del escenario.